

 **练习题 5.10** 作为另一个具有潜在的加载-存储相互影响的代码，考虑下面的函数，它将一个数组的内容复制到另一个数组：

```
1 void copy_array(long *src, long *dest, long n)
2 {
3     long i;
4     for (i = 0; i < n; i++)
5         dest[i] = src[i];
6 }
```

假设 *a* 是一个长度为 1000 的数组，被初始化为每个元素 *a*[*i*] 等于 *i*。

- 调用 `copy_array(a+1,a,999)` 的效果是什么？
- 调用 `copy_array(a,a+1,999)` 的效果是什么？
- 我们的性能测试表明问题 A 调用的 CPE 为 1.2(循环展开因子为 4 时，该值下降到 1.0)，而问题 B 调用的 CPE 为 5.0。你认为是什么因素造成了这样的性能差异？
- 你预计调用 `copy_array(a,a,999)` 的性能会是怎样的？

 **练习题 5.11** 我们测量出前置和函数 `psum1`(图 5-1) 的 CPE 为 9.00，在测试机器上，要执行的基本操作——浮点加法的延迟只是 3 个时钟周期。试着理解为什么我们的函数执行效果这么差。

下面是这个函数内循环的汇编代码：

Inner loop of psum1

a in %rdi, i in %rax, cnt in %rdx

1	.L5:	loop:
2	<code>vmovss -4(%rsi,%rax,4), %xmm0</code>	<i>Get p[i-1]</i>
3	<code>vaddss (%rdi,%rax,4), %xmm0, %xmm0</code>	<i>Add a[i]</i>
4	<code>vmovss %xmm0, (%rsi,%rax,4)</code>	<i>Store at p[i]</i>
5	<code>addq \$1, %rax</code>	<i>Increment i</i>
6	<code>cmpq %rdx, %rax</code>	<i>Compare i:cnt</i>
7	<code>jne .L5</code>	<i>If !=, goto loop</i>

参考对 `combine3`(图 5-14) 和 `write_read`(图 5-36) 的分析，画出这个循环生成的数据相关图，再画出计算进行时由此形成的关键路径。解释为什么 CPE 如此之高。

 **练习题 5.12** 重写 `psum1`(图 5-1) 的代码，使之不需要反复地从内存中读取 *p*[*i*] 的值。不需要使用循环展开。得到的代码测试出的 CPE 等于 3.00，受浮点加法延迟的限制。

5.13 应用：性能提高技术

虽然只考虑了有限的一组应用程序，但是我们能得出关于如何编写高效代码的很重要的经验教训。我们已经描述了许多优化程序性能的基本策略：

1) 高级设计。为遇到的问题选择适当的算法和数据结构。要特别警觉，避免使用那些会渐进地产生糟糕性能的算法或编码技术。

2) 基本编码原则。避免限制优化的因素，这样编译器就能产生高效的代码。

- 消除连续的函数调用。在可能时，将计算移到循环外。考虑有选择地妥协程序的模块性以获得更大的效率。
- 消除不必要的内存引用。引入临时变量来保存中间结果。只有在最后的值计算出来时，才将结果存放到数组或全局变量中。

3) 低级优化。结构化代码以利用硬件功能。

- 展开循环, 降低开销, 并且使得进一步的优化成为可能。
- 通过使用例如多个累积变量和重新结合等技术, 找到方法提高指令级并行。
- 用功能性的风格重写条件操作, 使得编译采用条件数据传送。

最后要给读者一个忠告, 要警惕, 在为了提高效率重写程序时避免引入错误。在引入新变量、改变循环边界和使得代码整体上更复杂时, 很容易犯错误。一项有用的技术是在优化函数时, 用检查代码来测试函数的每个版本, 以确保在这个过程没有引入错误。检查代码对函数的新版本实施一系列的测试, 确保它们产生与原来一样的结果。对于高度优化的代码, 这组测试情况必须变得更加广泛, 因为要考虑的情况也更多。例如, 使用循环展开的检查代码需要测试许多不同的循环界限, 保证它能够处理最终单步迭代所需要的所有不同的可能的数字。

5.14 确认和消除性能瓶颈

至此, 我们只考虑了优化小的程序, 在这样的小程序中有一些很明显限制性能的地方, 因此应该是集中注意力对它们进行优化。在处理大程序时, 连知道应该优化什么地方都是很难的。本节会描述如何使用代码剖析程序(code profiler), 这是在程序执行时收集性能数据的分析工具。我们还展示了一个系统优化的通用原则, 称为 Amdahl 定律(Amdahl's law), 参见 1.9.1 节。

5.14.1 程序剖析

程序剖析(profiling)运行程序的一个版本, 其中插入了工具代码, 以确定程序的各个部分需要多少时间。这对于确认程序中我们需要集中注意力优化的部分是很有用的。剖析的一个有力之处在于可以在现实的基准数据(benchmark data)上运行实际程序的同时, 进行剖析。

Unix 系统提供了一个剖析程序 GPROF。这个程序产生两种形式的信息。首先, 它确定程序中每个函数花费了多少 CPU 时间。其次, 它计算每个函数被调用的次数, 以执行调用的函数来分类。这两种形式的信息都非常有用。这些计时给出了不同函数在确定整体运行时间中的相对重要性。调用信息使得我们能理解程序的动态行为。

用 GPROF 进行剖析需要 3 个步骤, 就像 C 程序 prog.c 所示, 它运行时命令行参数为 file.txt:

1) 程序必须为剖析而编译和链接。使用 GCC(以及其他 C 编译器), 就是在命令行上简单地包括运行时标志 “-pg”。确保编译器不通过内联替换来尝试执行任何优化是很重要的, 否则就可能无法正确刻画函数调用。我们使用优化标志 -Og, 以保证能正确跟踪函数调用。

```
linux> gcc -Og -pg prog.c -o prog
```

2) 然后程序像往常一样执行:

```
linux> ./prog file.txt
```

它运行得会比正常时稍微慢一点(大约慢 2 倍), 不过除此之外唯一的区别就是它产生了一个文件 gmon.out。

3) 调用 GPROF 来分析 gmon.out 中的数据。

```
linux> gprof prog
```